DAY 4 DSA

Complete Lecture-Based Study Notes

Linked Lists • Reversal Problems • Stack & Queue • Expression Evaluation

Prepared from your uploaded Day 4 lecture transcript

Beginner-friendly explanations, code examples, dry runs, complexity analysis, interview definitions, and practice questions.

Source note: The transcript is the primary source. Code snippets and extra explanations are study aids added to make the concepts easier to practise; they are not represented as the instructor's exact original code.

How to Use These Notes

Read each topic in this order: **idea** → **pointer/operation logic** → **code** → **dry run** → **complexity** → **interview check**. For linked lists, draw the nodes and arrows while reading. For stack, queue, and expression questions, write down the state after every operation.

Lecture roadmap

• Doubly circular linked list: structure, insertion at the end, insertion at the beginning, traversal and pointer updates.

• Time complexity: compare operations that need traversal with operations that can use known pointers.

• Linked-list sorting: Selection Sort logic and its complexity.

• Reversing a singly linked list: iterative pointer reversal, recursive reversal, and reverse display.

• Interview problems: middle node, coding-round habits, edge cases and dry runs.

• Stack and Queue: definitions, operations, array-based implementations and use cases.

• Infix, prefix and postfix expressions; evaluating a postfix expression with a stack.

> The lecture moves from linked-list revision and tricky interview problems to stack and queue fundamentals, then expression notation and postfix evaluation.

1. Linked-List Refresher

What is a linked list?

A linked list is a sequence of nodes. Each node stores data and one or more links (pointers/references) to other nodes. Unlike an array, its nodes do not have to occupy consecutive memory locations.

```
[data | next] -> [data | next] -> [data | next] -> null
```

Head refers to the first node. In a singly linked list, each node has a next link. In a doubly linked list, each node has both next and previous links.

Singly vs doubly vs circular

Type	Links per node	End condition
Singly linked	next	last.next = null
Doubly linked	prev and next	first.prev = null; last.next = null
Circular singly	next	last.next = head
Doubly circular	prev and next	last.next = head and head.prev = last

Doubly circular linked list

A doubly circular linked list combines the two-way links of a doubly linked list with the wrap-around links of a circular list.

```
        +--------------------------------------+
        |                                      v
head -> [10] <-> [20] <-> [30] <-> [40] ------+
 ^                                            |
 +--------------------------------------------+
Conceptually:
head.prev = tail
tail.next = head
```

For the example above: 10.prev is 40, 40.next is 10, and every middle node has its normal previous and next neighbours.

> **Important:** because the structure is circular, traversal does not end at null. A traversal loop must stop when it returns to the starting node; otherwise it can run forever.

2. Doubly Circular Linked List: Pointer Updates

Adding a node at the end

The lecture highlights a useful trick: in a doubly circular list, the last node can be reached directly through **head.prev**. Therefore, if head is valid, you do not need to traverse from head to discover the tail.

Suppose the current list is 10 ■ 20 ■ 30 ■ 40, and a new node containing 50 must be appended.

Pointer changes

• Let tail = head.prev.

• Set newNode.prev = tail.

• Set newNode.next = head.

• Set tail.next = newNode.

• Set head.prev = newNode.

```
tail = head.prev
newNode.prev = tail
newNode.next = head
tail.next = newNode
head.prev = newNode
```

The four link changes reconnect both directions. The new node becomes the tail because the old tail now points forward to it, while the new node points back to the old tail.

Dry run

Before	tail=40; head=10; 40.next=10; 10.prev=40
New node	newNode=50
After	40.next=50; 50.prev=40; 50.next=10; 10.prev=50

After insertion, the circular order is 10 → 20 → 30 → 40 → 50 → 10. The backward links are consistent as well.

> **Complexity:** O(1) for insertion at the end when head is available and head.prev correctly identifies the tail. If the implementation has no way to access the tail directly, finding the last node first would require O(n) traversal.

Adding a node at the beginning

In a doubly circular list, inserting before the current head requires reconnecting the new node, the old tail, and the old head. If the list is non-empty:

```
tail = head.prev
newNode.prev = tail
newNode.next = head
tail.next = newNode
head.prev = newNode
head = newNode
```

The first five link assignments are similar to end insertion; the final step changes which node the head variable refers to. Handle the empty-list case separately: a common representation makes the new node point to itself in both directions.

A frequent bug is changing only next links and forgetting previous links. In a doubly linked list, each connection must remain consistent in both directions.

Displaying a circular list

```
if head == null:
    print("Empty")
else:
    current = head
    do:
        print(current.data)
        current = current.next
    while current != head
```

This pseudocode uses a do-while pattern because a non-empty circular list should print its first node before checking whether it has returned to head. An equivalent while-loop can be used if its first-node handling is correct.

3. Linked-List Operation Complexity

Time complexity describes how the amount of work grows as the number of nodes n grows. It does not measure exact seconds.

Operation	Typical time	Reason
Insert at beginning	O(1)	Only a fixed number of pointer changes.
Insert at end of doubly circular list	O(1)	head.prev gives the tail directly.
Search for a value	O(n)	May inspect every node.
Display/traverse all nodes	O(n)	Each node is visited.
Access the kth node	O(n)	Must follow links from a known node.
Delete a known node in doubly linked list	O(1)	Can reconnect its neighbours if node reference is already known.
Delete by value	O(n)	Search may be needed before deletion.

Key distinction: insertion can be O(1) when the insertion position is already known. Finding that position can still cost O(n). Complexity claims must state what pointers/references the operation is given.

4. Sorting a Linked List with Selection Sort

Selection Sort repeatedly finds the smallest value in the unsorted part and places it at the beginning of that part. The lecture discusses sorting a linked list using the Selection Sort idea.

Example

Input: 30 → 10 → 40 → 20

Pass 1: smallest is 10; put it first → 10 → 30 → 40 → 20

Pass 2: smallest among 30, 40, 20 is 20 → 10 → 20 → 40 → 30

Pass 3: smallest among 40, 30 is 30 → 10 → 20 → 30 → 40

Simple value-swapping pseudocode

```
current = head
while current is not null:
    minNode = current
    scan = current.next

    while scan is not null:
        if scan.data < minNode.data:
            minNode = scan
        scan = scan.next

    swap(current.data, minNode.data)
    current = current.next
```

This version swaps node data, which is easier to understand. Another design rearranges node links, but pointer-relinking is more error-prone and needs careful handling of head and neighbouring nodes.

Time complexity: O(n²), because for each node the algorithm scans the remaining unsorted nodes. **Auxiliary space:** O(1) for the iterative version above.

> **Stability note:** standard Selection Sort is generally not stable when it swaps values/nodes across the list. Stability matters when equal-key records must retain their original order.

5. Reversing a Singly Linked List: Iterative Method

Reversing a linked list means changing the direction of every next link. For example, 10 → 20 → 30 → null becomes 30 → 20 → 10 → null.

Three-pointer approach

Use three references: **previous**, **current**, and **nextNode**. The most important rule is to save current.next before changing it; otherwise the remaining list can become unreachable.

```
previous = null
current = head

while current != null:
    nextNode = current.next   # save the rest of the list
    current.next = previous   # reverse this link
    previous = current        # advance previous
    current = nextNode        # advance current

head = previous
```

Dry run: 10 → 20 → 30 → null

Step	previous	current	Action
Start	null	10	No links changed.
1	10	20	10.next becomes null.
2	20	30	20.next becomes 10.
3	30	null	30.next becomes 20.
Finish	30	null	Set head = 30.

Result: 30 → 20 → 10 → null.

Time: O(n), because each node is processed once. **Extra space:** O(1), because only a few references are used.

Common mistakes

• Changing current.next before saving the old next pointer.

• Moving current to current.next after reversing the link; current.next now points backwards.

• Forgetting to assign head = previous after the loop.

• Not testing an empty list and a one-node list.

6. Reversing a Linked List Using Recursion

Recursion solves a problem by calling the same function on a smaller subproblem. For linked-list reversal, the base case is reached when the list is empty or contains one node.

```
reverse(head):
    if head == null OR head.next == null:
        return head

    newHead = reverse(head.next)
    head.next.next = head
    head.next = null
    return newHead
```

How the recursive logic works

For 10 → 20 → 30 → null, the recursive calls go down until node 30 is reached. That node becomes the new head. As the calls return, each node points its successor back to itself: 30.next = 20, then 20.next = 10. Setting head.next = null breaks the old forward link and prevents a cycle.

Base case: empty or one-node list; already reversed. **Recursive case:** reverse the remainder, then attach the current node after its former successor.

Time: O(n). **Extra space:** O(n) call-stack space in the worst case. Deep lists can cause stack overflow in languages/runtimes with limited recursion depth.

> Do not confuse the new head returned from the recursive call with the original head. The original head becomes the final node.

7. Reverse Display vs Actually Reversing

Sometimes an interviewer asks to display a linked list in reverse order, not to modify the links.

Using recursion to print backwards

```
printReverse(node):
    if node == null:
        return
    printReverse(node.next)
    print(node.data)
```

For 10 → 20 → 30, the calls reach null first; printing occurs while calls return, so output is 30, 20, 10. The links are unchanged.

Complexity: O(n) time and O(n) recursion space. The distinction is important: *reverse the list* changes links; *print in reverse* only changes output order.

8. Finding the Middle Node

A common interview solution uses the slow-and-fast pointer technique. Slow moves one node per iteration; fast moves two. When fast reaches the end, slow is around the middle.

```
slow = head
fast = head

while fast != null AND fast.next != null:
    slow = slow.next
    fast = fast.next.next

return slow
```

Dry run

For 10 → 20 → 30 → 40 → 50: after one iteration slow=20 and fast=30; after two, slow=30 and fast=50; the next check stops because fast.next is null. The middle is 30.

For an even number of nodes, this common loop returns the second of the two middle nodes. If the interviewer expects the first middle, the loop condition or initial pointer setup must be adjusted.

Time: O(n). **Extra space:** O(1).

9. Interview and Coding-Round Habits

The lecture emphasizes practising the algorithms yourself instead of only watching the solution. In a coding round, explain the approach, code carefully, and test boundary cases.

Before coding

• Clarify the input: can the list be empty? Can it contain one node? Are duplicates allowed?

• State the approach before writing code.

• Identify which pointer/reference represents the head and whether the tail is available.

• Mention expected time and auxiliary-space complexity.

After coding: test edge cases

• Empty list: head = null.

• Single node: one node points to null (linear list) or to itself (circular list).

• Two nodes: exposes many reversal and deletion mistakes.

• Head/tail operations: verify both forward and backward links for doubly linked lists.

• Repeated values: confirm sorting/searching behaves correctly.

• Circular traversal: ensure the loop stops after returning to head.

> A useful interview explanation format is: **approach** → **why it works** → **complexity** → **edge cases**.

10. Stack Fundamentals

A stack is a linear data structure that follows **LIFO: Last In, First Out**. The most recently inserted item is removed first. Think of a stack of plates: you usually take the top plate first.

```
Top
 |
 v
[30]  <- most recently pushed
[20]
[10]
```

Core operations

Operation	Meaning	Typical time
push(x)	Insert x at the top	O(1)
pop()	Remove and return the top item	O(1)
peek()/top()	Read the top without removing it	O(1)
isEmpty()	Check whether no elements exist	O(1)

Stack dry run

Start empty → push(10) gives [10] → push(20) gives [10, 20] with 20 at the top → push(30) gives [10, 20, 30] → pop() returns 30 and leaves [10, 20].

Overflow and underflow

• **Overflow:** trying to push into a fixed-size array stack that is already full.

• **Underflow:** trying to pop or peek when the stack is empty.

Array implementation pseudocode

```
top = -1

push(x):
    if top == capacity - 1:
        report overflow
    else:
        top = top + 1
        array[top] = x

pop():
    if top == -1:
        report underflow
    else:
        x = array[top]
        top = top - 1
        return x
```

For a stack backed by a dynamic array or linked list, implementation details differ, but push/pop are typically O(1) amortized/constant time under the usual assumptions.

11. Queue Fundamentals

A queue follows **FIFO: First In, First Out**. The earliest inserted item is removed first, like a line of people waiting for service.

```
Front -> [10] [20] [30] <- Rear
dequeue removes 10
enqueue(40) adds at rear
```

Core operations

Operation	Meaning	Typical time with a good implementation
enqueue(x)	Insert x at the rear	O(1)
dequeue()	Remove from the front	O(1)
front()/peek()	Read the front item	O(1)
isEmpty()	Check whether empty	O(1)

Queue dry run

enqueue(10) → [10]; enqueue(20) → [10, 20]; enqueue(30) → [10, 20, 30]; dequeue() returns 10 and leaves [20, 30].

If a queue is implemented by shifting every array element after each dequeue, dequeue can cost O(n). A circular array or linked-list implementation avoids that repeated shifting and supports O(1) queue operations.

Stack vs Queue

Property	Stack	Queue
Ordering rule	LIFO	FIFO
Insert	push at top	enqueue at rear
Remove	pop from top	dequeue from front
Examples	undo, call stack, expression evaluation	waiting lines, task scheduling, breadth-first search

12. Expression Notations

An arithmetic expression can be written in different notations depending on where the operator appears.

Notation	Rule	Example for (A + B) * C
Infix	Operator between operands	(A + B) * C
Prefix	Operator before operands	* + A B C
Postfix	Operator after operands	A B + C *

Why postfix/prefix are useful

In infix notation, precedence and parentheses may be needed to determine the intended order. Prefix and postfix encode the operator placement so that a valid expression can be evaluated with a stack without needing the same parentheses/precedence handling during evaluation.

Operator reminder

For common arithmetic operators, multiplication/division generally have higher precedence than addition/subtraction. Parentheses can override precedence. When converting expressions, also consider associativity for operators of equal precedence.

13. Evaluating a Postfix Expression with a Stack

For postfix evaluation, scan tokens from left to right. Push operands. When an operator appears, pop the required operands, apply the operator in the correct order, and push the result.

Algorithm

• Create an empty stack.

• Read the next token from left to right.

• If it is an operand, push its value.

• If it is a binary operator, pop rightOperand first, then leftOperand.

• Compute leftOperand operator rightOperand and push the result.

• After all tokens are processed, the stack should contain one result for a valid expression.

> **Operand order matters.** For subtraction and division, if the stack gives you 8 and then 3, the intended expression may be 3 – 8 or 3 ÷ 8, depending on the pop order. The first popped item is the right operand.

Worked example: 5 2 3 * +

This postfix expression means 5 + (2 × 3), so the answer is 11.

Token	Action	Stack (bottom → top)
5	Operand: push 5	[5]
2	Operand: push 2	[5, 2]
3	Operand: push 3	[5, 2, 3]
*	Pop 3 and 2; compute 2 × 3 = 6; push 6	[5, 6]
+	Pop 6 and 5; compute 5 + 6 = 11; push 11	[11]

Final stack has one value: **11**.

Pseudocode

```
for token in postfix_expression:
    if token is an operand:
        push(token)
    else:
        right = pop()
        left = pop()
        result = apply(token, left, right)
        push(result)

answer = pop()
```

For n tokens, evaluation is O(n) time and O(n) worst-case stack space. This assumes tokens are already separated/recognized and each operation takes constant time.

14. Prefix Evaluation: Core Difference

Prefix evaluation scans from right to left. Push operands. When an operator is found, pop the first operand and then the second operand, compute the operation in the correct order, and push the result.

For example, prefix * + 5 2 3 represents (5 + 2) × 3 = 21. When scanning from right to left, the stack first receives 3, then 2, then 5; + combines 5 and 2 to make 7, and * combines 7 and 3 to make 21.

> The scan direction and pop order differ from postfix. Practise a few examples rather than memorizing only the words.

15. Quick Interview Definitions

Q: What is a doubly circular linked list?
A: A linked list in which each node has previous and next links, the last node points forward to the head, and the head points backward to the last node.

Q: Why can end insertion be O(1) in a doubly circular list?
A: Because head.prev gives direct access to the tail, so traversal to find the last node is unnecessary.

Q: What is Selection Sort?
A: A comparison sorting algorithm that repeatedly selects the minimum element from the unsorted part and places it at the beginning of that part.

Q: How do you reverse a singly linked list iteratively?
A: Use previous, current and next references; save the next node, reverse current.next, then advance the references.

Q: How can the middle of a linked list be found in one pass?
A: Use slow and fast pointers; slow advances one step and fast two steps.

Q: What is a stack?
A: A linear data structure following LIFO, where insertion and removal occur at the top.

Q: What is a queue?
A: A linear data structure following FIFO, where insertion occurs at the rear and removal at the front.

Q: What is postfix notation?
A: A notation in which an operator appears after its operands, such as A B +.

Q: Why is a stack used for postfix evaluation?
A: Operands are stored until an operator arrives; the operator pops the required operands, computes the result, and pushes it back.

16. Practice Questions

Try these without looking at the answers first.

1. In a doubly circular linked list, what does head.prev refer to?
2. What is the time complexity of searching for a value in an unsorted linked list?
3. Reverse 4 → 7 → 9 → null using the iterative pointer method. What is the final head?
4. What are the time and auxiliary-space complexities of iterative linked-list reversal?
5. What is the difference between reversing a list and printing it in reverse order?
6. For 1 → 2 → 3 → 4, which node does the usual slow/fast-pointer loop return as the middle?
7. A stack receives push(4), push(8), push(2), then pop(). What value is returned?
8. A queue receives enqueue(4), enqueue(8), enqueue(2), then dequeue(). What value is returned?
9. Convert (A + B) * C to prefix and postfix.
10. Evaluate postfix 8 2 / 3 +. Show the stack after every token.
11. Why must the old next pointer be saved before reversing current.next?
12. What bug can occur if a circular linked-list traversal waits for current == null?

Answer key

1. The tail/last node.
2. O(n).
3. 9 → 7 → 4 → null; head becomes the node containing 9.
4. O(n) time and O(1) extra space.
5. Reversing changes links; reverse printing leaves links unchanged and changes only output order.
6. Node 3, the second middle node.
7. 2.
8. 4.
9. Prefix: * + A B C; postfix: A B + C *.
10. 8 → [8]; 2 → [8,2]; / → [4]; 3 → [4,3]; + → [7].
11. Otherwise the rest of the list may become unreachable after current.next is redirected.
12. In a circular list, traversal may never encounter null and can loop forever.

17. Last-Minute Revision Sheet

Topic	Remember
Doubly circular	head.prev = tail; tail.next = head
Append at end	Use tail=head.prev; update newNode.prev, newNode.next, tail.next, head.prev
Selection Sort	Repeated minimum selection; $O(n^2)$ time
Iterative reverse	previous/current/next; O(n) time, O(1) extra space
Recursive reverse	Base case: null or one node; O(n) call-stack space
Reverse display	Recurse first, print while returning; does not change links
Middle node	Slow one step, fast two steps; O(n) time, O(1) space
Stack	LIFO; push/pop at top
Queue	FIFO; enqueue at rear, dequeue at front
Postfix evaluation	Scan left to right; pop right then left; push result
Prefix evaluation	Scan right to left; be careful with operand order

Study suggestion: first reproduce the iterative reverse and postfix-evaluation algorithms from memory. Then explain every pointer/stack change aloud as if answering an interviewer.